

技術アーキテクチャー と AI-DLC の 実践と工夫

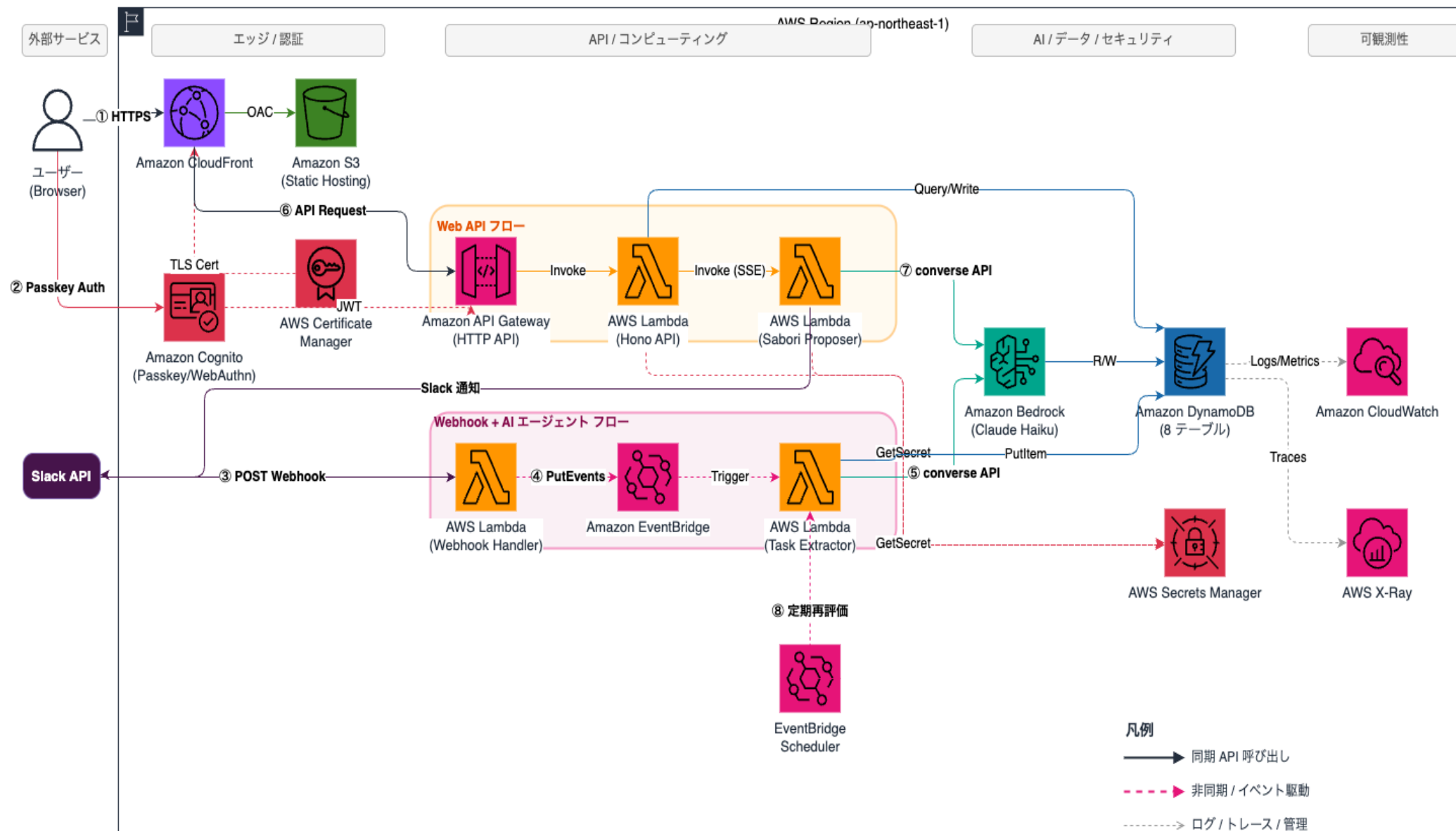
SABOROU — AWS Summit Japan 2026 Hackathon

技術アーキテクチャー

心理学 × AI Agents × フルサーバーレス

サボロー — AWS System Architecture

v2.0 | 2026-05 | ap-northeast-1 | AWS Summit Hackathon 2026

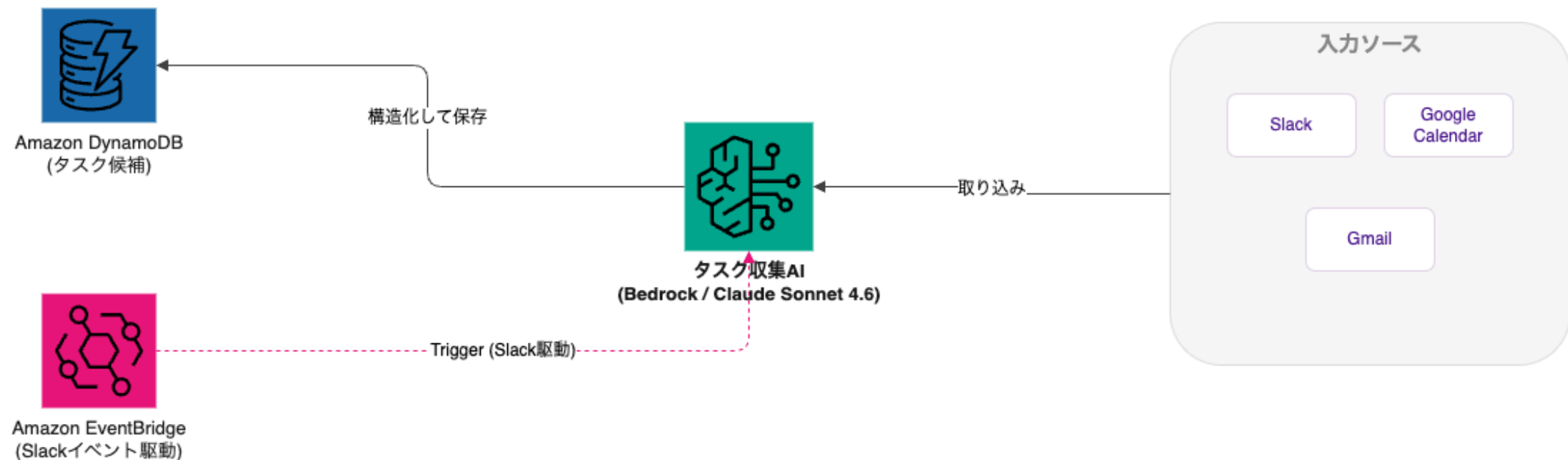


①~⑧: プレゼン説明順序

① AIエージェント

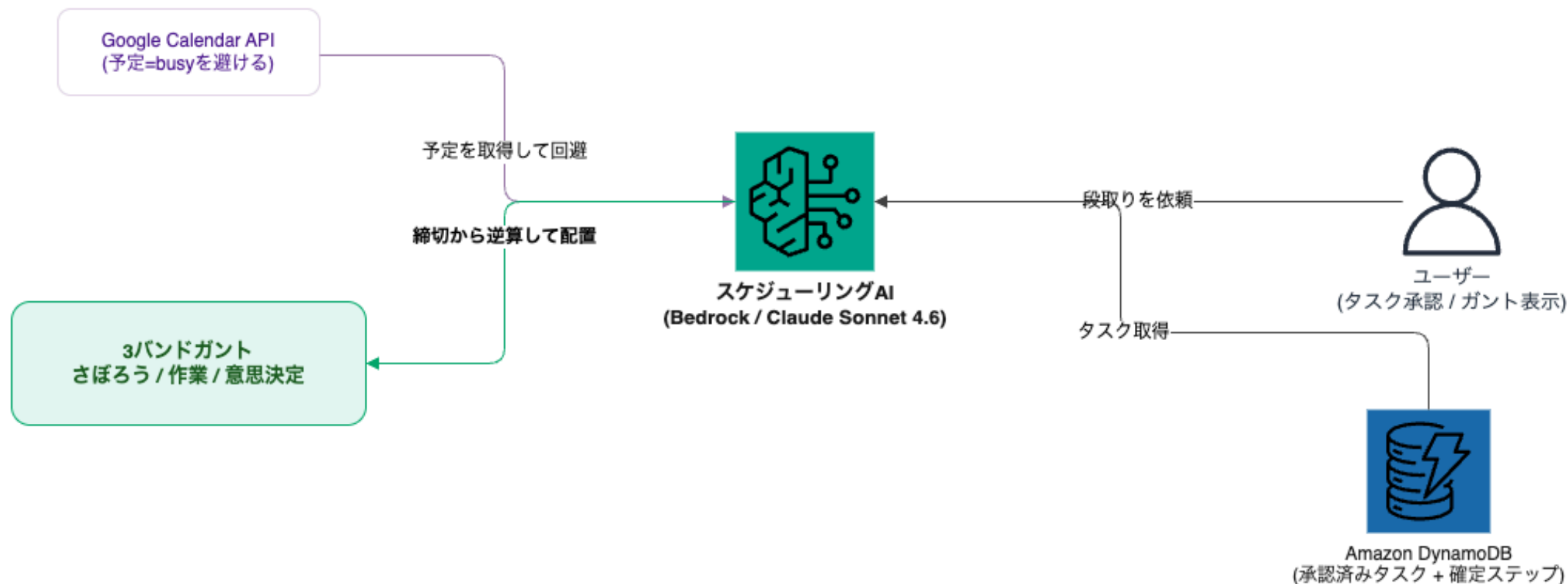
機能詳細：タスク収集AI

Slack / カレンダー / メールから、タスク・締切・関係者を文脈ごと収集して構造化する



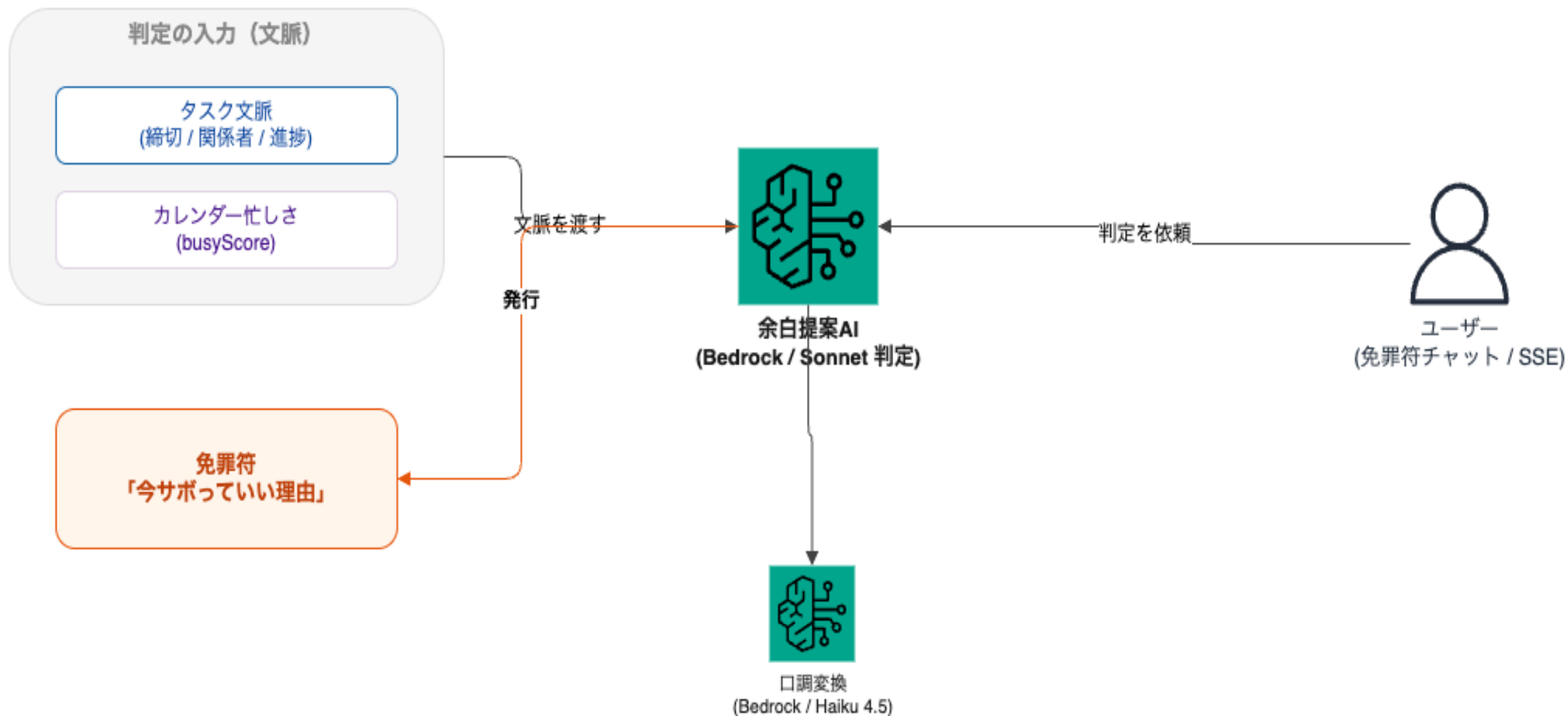
機能詳細：スケジューリングAI

タスクを作業ステップに分解し、締切から逆算して3バンドガントに配置する（意思決定は時刻アンカー）



機能詳細：余白提案AI

ガントの余白（サボれる時間）について「今サボっていい理由」を免罪符として発行する



3 AI Agents がつくる「サボり判定エンジン」

Agent ① 抽出

TaskExtractorAgent

Slack Webhookを受信し
Bedrock Tool Use で
タスクを**構造化抽出**

Agent ② 整理

TaskOrganizerAgent

依存関係を解析し
心理学5理論で
サボれる度スコアを算出

Agent ③ 提案

SaboriProposerAgent

根拠付きのサボり提案を生成
PersonaRenderer で
おっとり口調に変換

EventBridge でイベント駆動連携 — Slack Webhook から
SSE ストリーミング配信まで、3 Agents が**疎結合で一気通貫**

Bedrock Tool Use で「説明できるAI判定」を実現

構造化出力を強制する設計

```
// Tool Use で LLM の出力形式を保証
toolChoice: { type: "tool",
               name: "sabori_judgment" }
```

判定結果は必ずこの4フィールドで返る:

フィールド	内容
<code>verdict</code>	skip / delay / do_it
<code>reasoning</code>	判断の根拠（自然言語）
<code>summaryText</code>	1行サマリ
<code>nextCheckOffset</code>	次に確認する分数

AIが**何を根拠に判断したか**をユーザーに開示。
ブラックボックスにしない設計

心理学5理論をContextSignalに変換

Collective Effort Model—文脈欠損を検出

Identifiability Effect—依頼者の監視度を評価

Self-Determination Theory-外発プレッシャーを定量化

Expectancy Theory—今やる期待値を算出

Sucker Effect—損な役回りを検出

5理論すべてに査読論文 DOI あり

プロダクション品質のサーバーレスインフラ

AWS CDK v2 — 6スタック構成

CognitoStack 認証・
Google OAuth

DataStack DynamoDB 8
テーブル

ApiStack Hono on
Lambda

AgentStack 3 AI Agents

WebhookStack
EventBridge

FrontendStack
CloudFront+S3

Lambda

DynamoDB

Bedrock

EventBridge

Cognito

CDK v2

Lambda = サーバーレスでコスト最適 /

DynamoDB = On-Demand で0→スケール対応

Bedrock = モデルガバナンスとTool Use強制 /

EventBridge = Agent間の疎結合

品質の証拠

542

テスト全パス

0

cdk-nag エラー

\$31

月額コスト

Cognito PKCE + **HMAC-SHA256** 署名付き OAuth state でCSRF対策済み。Slack Webhook 署名検証。**外部データは処理後即削除**

AI-DLC の工夫

ただ使うのではなく、強化した

工夫 1 — Inception 前に環境を整え、AIをロジック設計に集中させた

AI-DLC 開始前にやったこと

- ① **モノレポ構築** (pnpm workspaces)
7 Unit の依存順序を先に設計
- ② **CI / テスト環境を先行整備**
Vitest ・ Playwright ・ Biome を設定済みに
- ③ **チームでゴールイメージを徹底すり合わせ**
要件 ・ 設計 ・ ユーザーストーリーを何度も確認

効果

AIへの「ノイズ」を排除

環境整備を人間側で実施して開始することで、
AIが**アーキテクチャ設計とロジック実装だけに集中**できる状態を作った

人間が何度も確認・修正した →

要件 ・ 設計 ・ ストーリーを繰り返しレビューし
チームで**合意してから**実装に進んだ

工夫 2 — 形式検証をワークフローに追加し、安全性を通学的に証明した

証明した3つのコアロジック

GuardTokenLimit

— 二分探索の**終了性**と境界値安全性

Pseudonymize

— 仮名化の**冪等性**・SHA-256 決定性

ContextUtils

— スコアの**単調性**・ゼロ除算不発生

専用 SKILL + サブエージェントを自作

Inception Application Design 完了後、**実装前に** Leanで定理を証明してから Construction へ進むフローを AI-DLC に組み込んだ

オフバイワン証明（核心）

```
-- 通常の mid だと low=h-1 で無限ループ!  
-- 上側バイアス式で終了を数学的に保証:  
def upperBiasedMid (low high : Nat) :=  
  (low + high + 1) / 2  
  
theorem upperBiasedMid_gt_low  
  (h : low < high) :  
    upperBiasedMid low high > low := by  
  simp [upperBiasedMid]; omega -- QED ✓
```

AIが設計したロジックの正しさを
数学的定理として証明してから実装

工夫 3 — AI-DLCのバグを発見し、公式OSSに PR してマージさせた

発見したバグ: `session-continuity.md` の不整合

6ユニット・Construction ランで実際に遭遇した問題:

問題① パスに `{unit-name}` が含まれない → ファイルが見つからない

問題② 複数ユニット時にどれを読むか指示がない

問題③ 実在しないファイル名を指定していた

修正の核心

`Read functional-design.md` を

`aidlc-docs/construction/{unit-name}/functional-design/` に修正

し、`aidlc-state.md` から進行中ユニットを特定するよう改善

awslabs / aidlc-workflows

PR #276

✓ MERGED

Approved by

harmjeff

mayakost

PRのリンクはこちら

aws

labs

/

aidlc-workflows

Q

Type ↵ to search

<>

Code

⌚

Issues

32

🔗

Pull requests

36

👤

Agents

▶

Actions

📁

Projects

🛡️

Security and quality

📈

Insights

fix: scope per-unit artifact loading to unit directories in session-continuity #276

Code ▾

🔗 Merged

mayakost merged 2 commits into aws:labs:main from Mameta29:fix/session-continuity-per-unit

📄 last week

💬 Conversation 6

🔗 Commits 2

📄 Checks 24

📄 Files changed 1

+6 -1

Mameta29 commented 2 weeks ago • edited ▾

Contributor ⋮

Summary

Closes #275.

The "Per-Unit Design" artifact-loading instruction in session-continuity.md did not reflect the actual per-unit directory structure of the Construction phase. A resumed, zero-context session had no way to know which unit's artifacts to load or where they live.

Problem

session-continuity.md line 33 read:

- **Per-Unit Design**: Read functional-design.md, nfr-requirements.md, nfr-design.md, infrastructure.g

This is inaccurate in three ways (verified against the current rule files):

1. Missing the {unit-name} directory level.

Every Construction stage writes to aidlc-docs/construction/{unit-name}/{stage}/ (e.g. functional-design.md lines 68-71). The line listed bare filenames with no path.

2. No unit selection for multi-unit projects.

Construction runs a per-unit loop; an N-unit project has N copies of these directories. The line gave no guidance on which unit(s) to load.

3. Filenames did not match the real artifacts.

There is no single functional-design.md — the stage produces functional-design/business-logic-model.md, business-rules.md, domain-entities.md; similarly for nfr-design and nfr-requirements.

The other lines in the same list are fine because those stages are not unit-scoped — only Per-Unit Design has artifacts under a {unit-name} directory.

Change

Reviewers

harmjeff

✓

mayakost

✓

scottschreckengau

🛡️ ●

spraja08

🛡️ ●

leandrodamascena

🛡️ ●

raj-jain-aws

🛡️ ●

Kalindi-Dev

🛡️ ●

Assignees

No one assigned

Labels

rules

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

🔗 [Bug]: session-continuity.md Per-Unit Design loa...

14